

Modeling and Optimizing a Battery Charging Profile

In this project, you will use MATLAB to model a battery charging profile. You will fit real lithium-ion battery data in MATLAB, compute energy transfer through numerical integration, and analyze key performance characteristics such as charge-rate behavior and efficiency losses.

Use this Live Script as a template to get started on your project solution. You may include helper functions at the bottom of this document or as separate files. For a tutorial on how to use Live Scripts, check out this [video](#).

Suggested Tasks

1. Given data for a lithium-ion battery charging profile, use MATLAB to fit the equation for voltage provided in the Project Description.
2. Plot the following curves: voltage vs. time, current vs. time, and power vs. time.
3. Use integration to compute the total energy delivered to the battery (area under power-time curve).
4. Analyze the rate of voltage change at different intervals (calculate derivatives), compute the time to reach 80% and 100% charge, and compute the energy lost due to resistance.

Table of Contents

Break Down the Problem	1
Task 1: Fit the voltage equation to a lithium-ion battery profile	1
Task 2: Plot the electrical terms that express how energy moves in a circuit	19
Task 3: Compute analytical results	22
Interpretation of Results	23

Break Down the Problem

In the space below, define the problem statement:

List the requirements, constraints, and success criteria for the project:

What is your proposed solution?

List the steps you will need to take to achieve your proposed solution:

What challenges do you face in the process of achieving the solution?

Task 1: Fit the voltage equation to a lithium-ion battery profile

Given real time-series voltage data for a lithium-ion battery, use MATLAB Curve Fitting Toolbox to fit the

equation $V(t) = V_{\max} \left(1 - e^{-\frac{t}{RC}} \right)$ and model the battery charging profile as a simple RC-circuit analog.

Load MathWorks example lithium-ion batteries data by following the steps [here](#). The page also includes information about the dataset, how the data was recorded, and the battery's maximum voltage.

Note: The battery from this example dataset has been cycled to failure (charged and discharged repeatedly). For this project, you may choose to use data from a single cycle and focus only on the period of time within a cycle that the battery is being charged. The battery is fully charged at 3.6V. This example battery uses a dynamic fast charging profile, but for this project, you will attempt to model the charging profile using a simple exponential capacitor charging equation.

Then, follow the workflow outlined in the [documentation](#) to fit the voltage equation to your selected data. Plot the data and fitted curve in the same figure and, finally, print the goodness-of-fit statistics.

```
% Insert your code in the spaces below (or make use of helper functions and
% indicate where
% they can be found). Ensure your code is well-documented.

% STEP 1a
% Load the example battery dataset, plus view of first 5 lines as preview
dataFile
= matlab.internal.examples.downloadSupportFile("predmaint","batteryagingdata/
singlecell/v1/singleCellLifeTimeData.zip");
unzip(dataFile)
load("singleCellLifeTimeData.mat")
head(data, 5)
```

Data_Point	Test_Time	DateTime	Step_Time	Step_Index	Cycle_Index	Current	Voltage
1	0	13-May-2017 03:21:40	-1780	0	0	0	3.3018
2	0.0001	13-May-2017 03:21:40	-1780	0	0	0	3.3018
3	9.9983	13-May-2017 03:21:50	-1770	0	0	0	3.3018
4	20.002	13-May-2017 03:22:00	-1760	0	0	0	3.3018
5	30.001	13-May-2017 03:22:10	-1750	0	0	0	3.3018

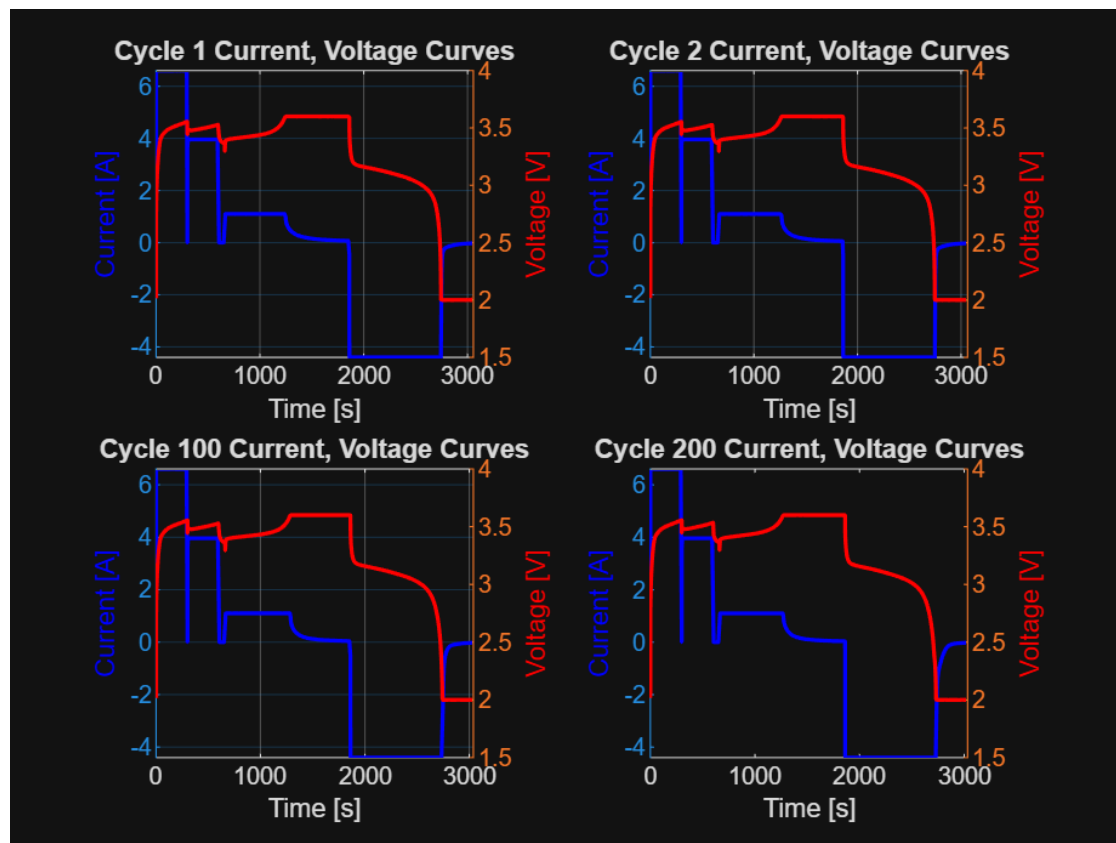
```
% STEP 1b
% Battery data sample evaluation: observing differences in the charging cycles over
% a select sample of cycles (just for team reference, this is not part of
% the task requirements/steps.

% multi-subplot generation to compare cycles 1, 2, 100, 200, 300, 400, 500, 600,
700, 800, 875, and 876
sample_cycles_set1 = [1, 2, 100, 200];
sample_cycles_set2 = 300:100:600;
sample_cycles_set3 = [700, 800, 875, 876];
```

```

figure;
for index = 1:length(sample_cycles_set1)
grid on;
subplot(2, 2, index)
cycleData = data(data.Cycle_Index == sample_cycles_set1(index), :);
cycleData.DateTime = seconds(cycleData.DateTime - cycleData.DateTime(1));
yyaxis left;
gca.YColor = 'blue';
plot(cycleData.DateTime,cycleData.Current, '-b', 'LineWidth', 1.5);
ylabel('Current [A]', 'Color', 'blue');
yyaxis right;
gca.YColor = 'red';
plot(cycleData.DateTime,cycleData.Voltage, '-r', 'LineWidth', 1.5);
ylabel('Voltage [V]', 'Color', 'red');
xlabel("Time [s]");
title(sprintf('Cycle %d Current, Voltage Curves', sample_cycles_set1(index)))
end

```



```

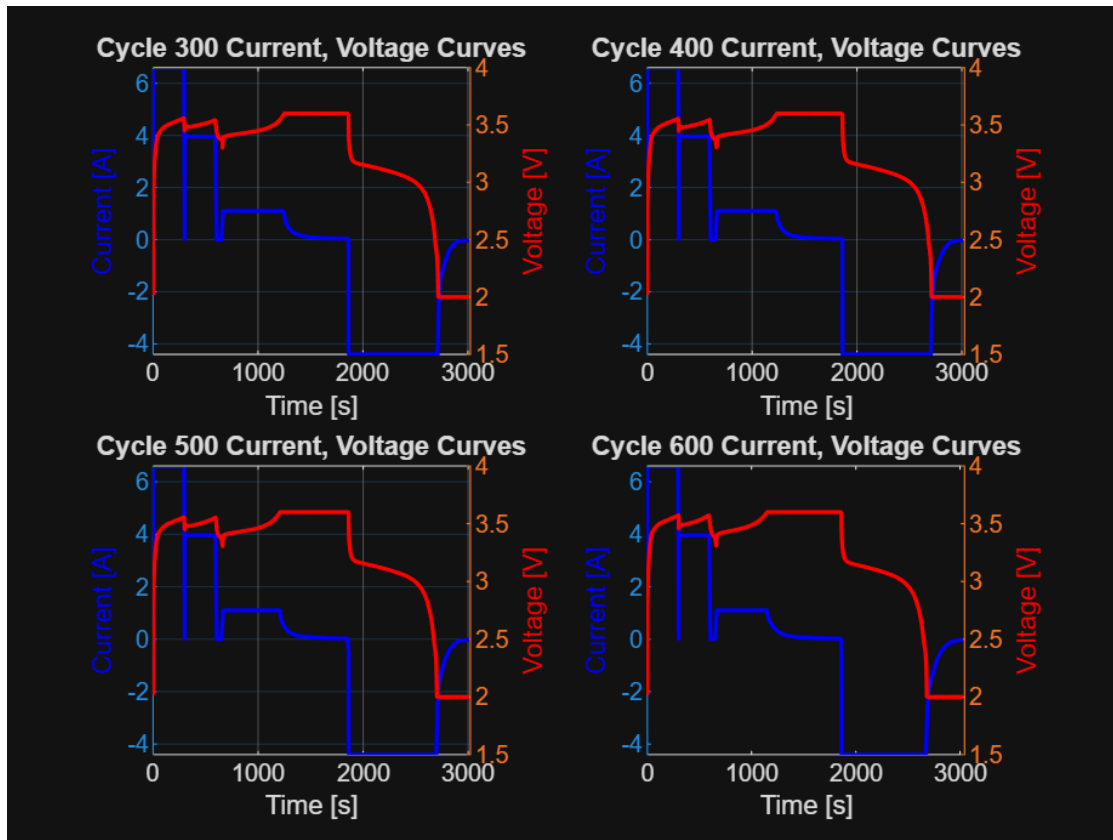
figure;
for index = 1:length(sample_cycles_set2)
grid on;
subplot(2, 2, index)
cycleData = data(data.Cycle_Index == sample_cycles_set2(index), :);
cycleData.DateTime = seconds(cycleData.DateTime - cycleData.DateTime(1));
yyaxis left;
gca.YColor = 'blue';

```

```

plot(cycleData.DateTime,cycleData.Current, '-b', 'LineWidth', 1.5);
ylabel('Current [A]', 'Color', 'blue');
yyaxis right;
gca.YColor = 'red';
plot(cycleData.DateTime,cycleData.Voltage, '-r', 'LineWidth', 1.5);
ylabel('Voltage [V]', 'Color', 'red');
xlabel("Time [s]");
title(sprintf('Cycle %d Current, Voltage Curves', sample_cycles_set2(index)))
end

```

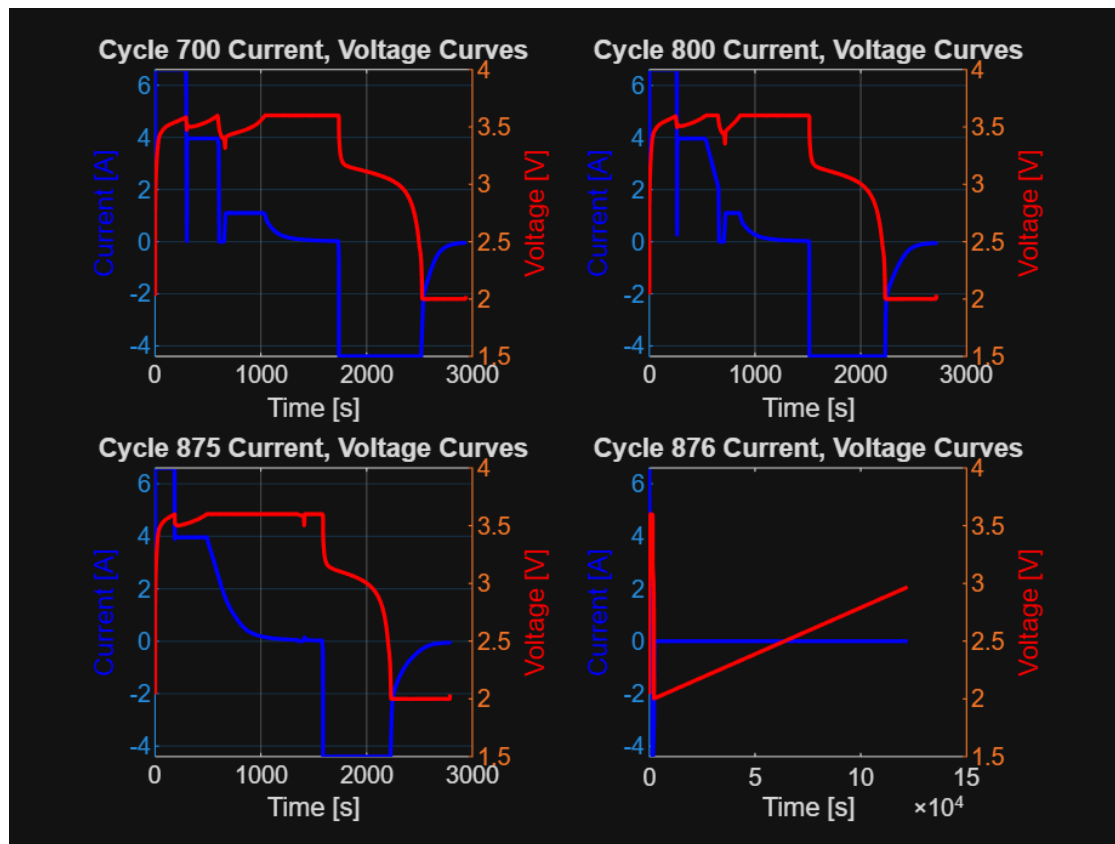


```

figure;
for index = 1:length(sample_cycles_set3)
grid on;
subplot(2, 2, index)
cycleData = data(data.Cycle_Index == sample_cycles_set3(index), :);
cycleData.DateTime = seconds(cycleData.DateTime - cycleData.DateTime(1));
yyaxis left;
gca.YColor = 'blue';
plot(cycleData.DateTime,cycleData.Current, '-b', 'LineWidth', 1.5);
ylabel('Current [A]', 'Color', 'blue');
yyaxis right;
gca.YColor = 'red';
plot(cycleData.DateTime,cycleData.Voltage, '-r', 'LineWidth', 1.5);
ylabel('Voltage [V]', 'Color', 'red');
xlabel("Time [s]");
title(sprintf('Cycle %d Current, Voltage Curves', sample_cycles_set3(index)))
end

```

end



```
% NOTE: the charging-discharging cycles start to degrade near the end...  
% choosing CYCLE 1 for the next part because it doesn't have as many...  
% discontinuities/gaps as some of the other randomly sampled cycle graphs.
```

```
% STEP 2  
% Pre-process the data to focus on a single cycle, and the period of time  
% within that cycle that the battery was being charged  
  
% Single cycle to focus on: CYCLE 1, code below is just from following the...  
% 'MathWorks example Li-ion batteries data'  
bParser = batteryTestDataParser(data);  
bParser.CurrentVariable = 'Current';  
bParser.VoltageVariable = "Voltage";  
bParser.TimeVariable = "DateTime";  
bParser.CycleIndexVariable = 'Cycle_Index';  
bParser.StepIndexVariable = 'Step_Index';  
  
% generate table formatting  
segmentedRawDataTable = segmentData(bParser);  
head(segmentedRawDataTable,5)
```

Data_Point	Test_Time	DateTime	Step_Time	Step_Index	Cycle_Index	Current	Voltage
------------	-----------	----------	-----------	------------	-------------	---------	---------

1	0	13-May-2017 03:21:40	-1780	0	0	0	3.3018
2	0.0001	13-May-2017 03:21:40	-1780	0	0	0	3.3018
3	9.9983	13-May-2017 03:21:50	-1770	0	0	0	3.3018
4	20.002	13-May-2017 03:22:00	-1760	0	0	0	3.3018
5	30.001	13-May-2017 03:22:10	-1750	0	0	0	3.3018

```
% STEP 3
% Follow the provided workflow to fit the voltage equation to your selected
% data.
% HINT: Vmax is known, so the only unknown in the equation is RC (also
% known as tau, the RC time constant - it is equivalent to the time it takes
% for a capacitor to reach ~63.2% of its max voltage).
% When you are fitting the curve to the data, you are essentially estimating
% the unknown parameter, tau.

% All the stuff below is just parsing the data to CONFIRM that Vmax ~= 3.6 Volts
% verify what variable 'dV/dt' is actually called
data.Properties.VariableNames
```

```
ans = 1x15 cell
'Data_Point' 'Test_Time' 'DateTime' 'Step_Time' 'Step_Index' 'Cycle_Index' 'C ...
```

```
% change var name of "dV/dt" because it isn't syntax-friendly to
% MATLAB ("data.dV/dt" keeps giving error")
data = renamevars(data, "dV/dt", "dVdt");

% boolean array of where dV/dt ~= 0, only in region of Vmax, only for cycle
% 1
mask = data.Cycle_Index == 1 & data.Voltage > 3.2 & abs(data.dVdt) < 0.01;

% use boolean array to analyze that flat region near the theoretical Vmax,
% in "Cycle 1 Current, Voltage Curves" graph
V_maxes_ls = data.Voltage(mask);

% temporarily displayed command: 'disp(Vmaxes_ls)', which is a list of Vmaxes
experimental_V_max = max(V_maxes_ls);

% just verifying if experimental_V_max is close to V_max = 3.6 Volts
disp(experimental_V_max)
```

```
3.6004
```

```
% STEP 4
% Plot the data and the fitted curve in the same figure.
% Using curve fitter! 't' is time (x-axis), 'V' is Voltage (y-axis), ...
% V_max is given 3.6 value.
% 'T' will be used to represent tau, the RC time constant (T = RC)
```

```
% checking data type, data values
data(data.Cycle_Index == 1, :)
```

```
ans = 1078x15 table
```

...

	Data_Point	Test_Time	DateTime	Step_Time	Step_Index
1	2834	1.0168e+04	14-May-2017 03:42:34	0.0123	12
2	2835	1.0168e+04	14-May-2017 03:42:34	0.0533	5
3	2836	1.0168e+04	14-May-2017 03:42:34	0.0798	5
4	2837	1.0168e+04	14-May-2017 03:42:35	0.0949	5
5	2838	1.0168e+04	14-May-2017 03:42:35	0.1087	5
6	2839	1.0168e+04	14-May-2017 03:42:35	0.1236	5
7	2840	1.0168e+04	14-May-2017 03:42:35	0.1374	5
8	2841	1.0168e+04	14-May-2017 03:42:35	0.1521	5
9	2842	1.0168e+04	14-May-2017 03:42:35	0.1653	5
10	2843	1.0169e+04	14-May-2017 03:42:35	0.1791	5
11	2844	1.0169e+04	14-May-2017 03:42:35	0.1957	5
12	2845	1.0169e+04	14-May-2017 03:42:35	0.2105	5
13	2846	1.0169e+04	14-May-2017 03:42:35	0.2246	5
14	2847	1.0169e+04	14-May-2017 03:42:35	0.2375	5

⋮

```
charging_data_only = data(data.Cycle_Index == 1 & data.Current >= 0, :);
```

```
% variables / parameters of (custom) fitting equation
```

```
t = seconds(charging_data_only.DateTime - charging_data_only.DateTime(1));
```

```
V = charging_data_only.Voltage;
```

```
V_max = 3.6;
```

```
% "optimization" of tau value?
```

```
model1 = @(T, t) V_max * (1 - exp(-t / T));
```

```
% placeholder guess for tau (T)
```

```
T0 = 1000;
```

```
T = lsqcurvefit(model1, T0, t, V);
```

Local minimum possible.

lsqcurvefit stopped because the final change in the sum of squares relative to its initial value is less than the value of the function tolerance.

<stopping criteria details>

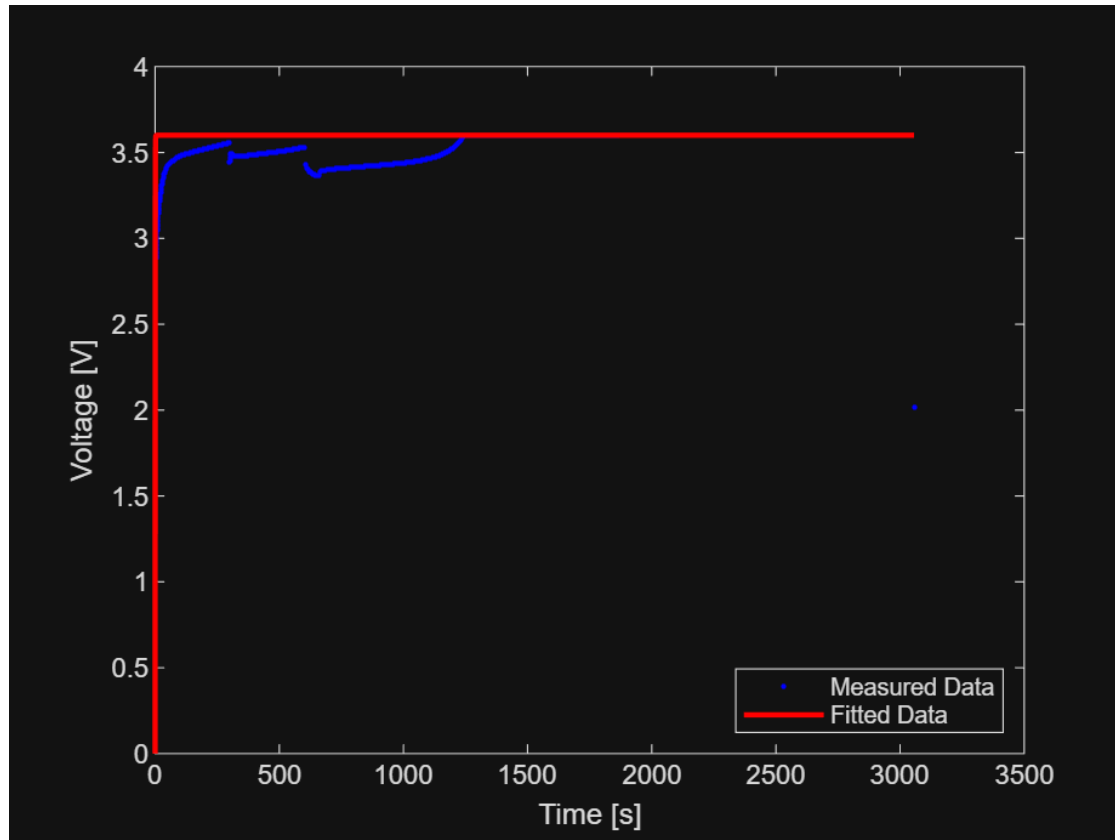
```

V_fit = model1(T, t);

% plotting both parsed data AND fitted data
figure
plot(t, V, 'b.')
hold on
plot(t, V_fit, 'r', 'LineWidth', 2)

% final graph labels + legend
xlabel('Time [s]')
ylabel('Voltage [V]')
legend('Measured Data', 'Fitted Data', 'Location', 'southeast')

```



```

% STEP 5
% Calc + display the goodness-of-fit statistics.
residuals = V - V_fit;

sum_of_squared_errors = sum(residuals.^2);

root_mean_squared_error = sqrt(mean(residuals.^2));

% next, use 'sum_of_squared_errors' and calc 'total sum of squares'
% to get R^2 (common statistical value)
total_sum_of_squares = sum((V - mean(V)).^2);

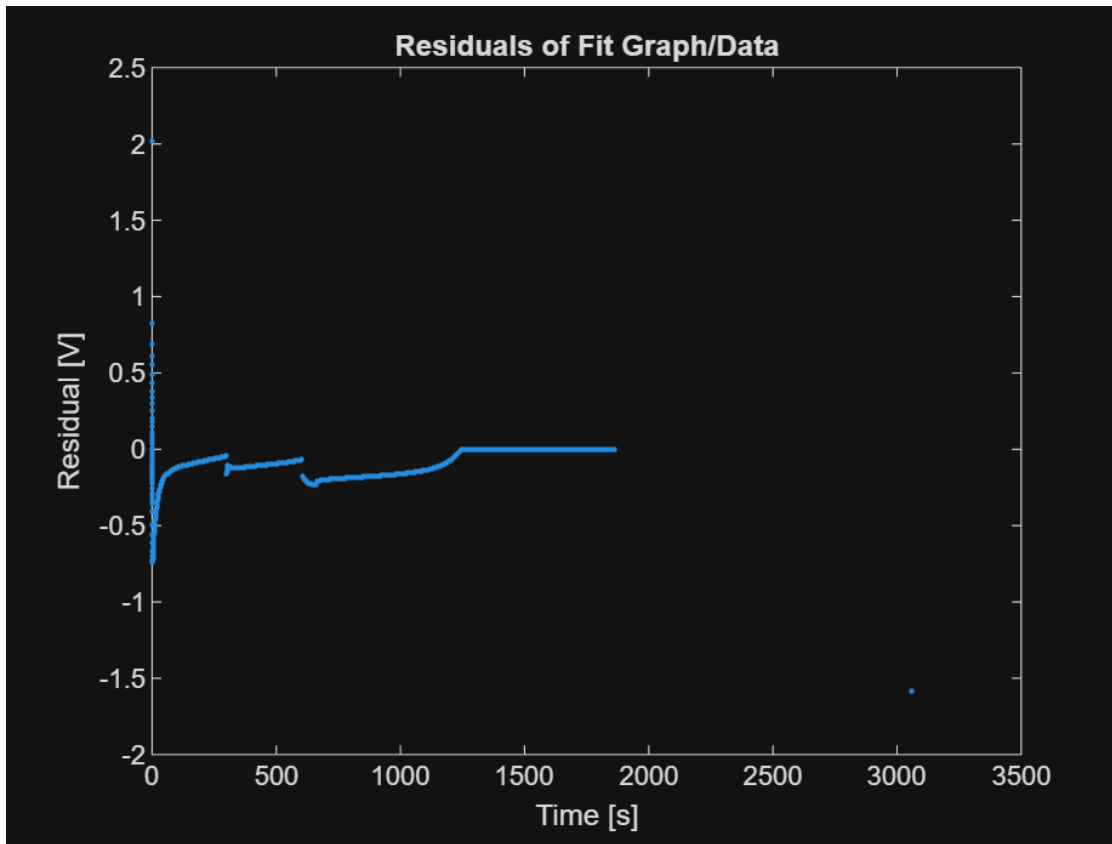
```

```

R2_value = 1 - sum_of_squared_errors / total_sum_of_squares;

% not sure if need to plot residuals because there's a lot of them...
% didn't realize it was an array
figure
grid on
plot(t, residuals, '.')
xlabel('Time [s]')
ylabel('Residual [V]')
title('Residuals of Fit Graph/Data')

```



```

% easy display statements
disp(residuals)

```

```

2.0153
0.8273
0.6876
0.6132
0.5574
0.4934
0.4388
0.3832
0.3382
0.3026
0.2486
0.2057
0.1784
0.1484
0.1127

```

0.0829
0.0588
0.0369
0.0095
-0.0131
-0.0309
-0.0535
-0.0615
-0.0765
-0.0958
-0.1104
-0.1260
-0.1444
-0.1527
-0.1702
-0.1849
-0.2002
-0.2180
-0.2283
-0.2591
-0.2896
-0.3012
-0.3290
-0.3538
-0.4070
-0.4937
-0.5577
-0.6164
-0.6660
-0.7013
-0.7225
-0.7350
-0.7404
-0.7406
-0.7367
-0.7308
-0.7235
-0.7235
-0.7151
-0.7059
-0.6964
-0.6865
-0.6764
-0.6664
-0.6561
-0.6459
-0.6359
-0.6258
-0.6158
-0.6158
-0.6057
-0.5955
-0.5852
-0.5748
-0.5648
-0.5543
-0.5442
-0.5342
-0.5239
-0.5137
-0.5035
-0.4932
-0.4832
-0.4731

-0.4629
-0.4525
-0.4423
-0.4321
-0.4321
-0.4220
-0.4119
-0.4018
-0.3918
-0.3918
-0.3817
-0.3717
-0.3615
-0.3514
-0.3414
-0.3312
-0.3212
-0.3212
-0.3111
-0.3010
-0.2909
-0.2808
-0.2707
-0.2707
-0.2607
-0.2507
-0.2507
-0.2406
-0.2306
-0.2205
-0.2105
-0.2005
-0.2005
-0.1903
-0.1803
-0.1711
-0.1636
-0.1636
-0.1571
-0.1513
-0.1513
-0.1460
-0.1409
-0.1362
-0.1317
-0.1317
-0.1275
-0.1275
-0.1238
-0.1203
-0.1171
-0.1171
-0.1141
-0.1141
-0.1112
-0.1112
-0.1088
-0.1088
-0.1062
-0.1039
-0.1017
-0.1017
-0.0995
-0.0975

-0.0955
-0.0955
-0.0934
-0.0914
-0.0898
-0.0879
-0.0861
-0.0861
-0.0843
-0.0843
-0.0826
-0.0809
-0.0792
-0.0792
-0.0775
-0.0775
-0.0758
-0.0742
-0.0742
-0.0724
-0.0707
-0.0707
-0.0691
-0.0674
-0.0674
-0.0656
-0.0641
-0.0641
-0.0624
-0.0606
-0.0589
-0.0589
-0.0572
-0.0572
-0.0553
-0.0536
-0.0519
-0.0500
-0.0482
-0.0464
-0.0446
-0.0443
-0.1579
-0.1477
-0.1374
-0.1264
-0.1163
-0.1058
-0.1096
-0.1096
-0.1155
-0.1187
-0.1187
-0.1206
-0.1216
-0.1221
-0.1221
-0.1221
-0.1217
-0.1213
-0.1207
-0.1207
-0.1201
-0.1192

-0.1184
-0.1176
-0.1176
-0.1168
-0.1160
-0.1151
-0.1142
-0.1135
-0.1126
-0.1116
-0.1107
-0.1107
-0.1098
-0.1091
-0.1091
-0.1080
-0.1072
-0.1062
-0.1062
-0.1053
-0.1053
-0.1043
-0.1043
-0.1035
-0.1035
-0.1026
-0.1017
-0.1006
-0.0999
-0.0989
-0.0979
-0.0979
-0.0971
-0.0960
-0.0960
-0.0950
-0.0950
-0.0940
-0.0930
-0.0930
-0.0920
-0.0920
-0.0909
-0.0898
-0.0898
-0.0888
-0.0888
-0.0877
-0.0866
-0.0854
-0.0854
-0.0842
-0.0842
-0.0830
-0.0817
-0.0803
-0.0792
-0.0792
-0.0778
-0.0764
-0.0764
-0.0750
-0.0750
-0.0735

-0.0735
-0.0720
-0.0703
-0.0689
-0.1734
-0.1734
-0.1900
-0.2015
-0.2015
-0.2100
-0.2100
-0.2164
-0.2213
-0.2252
-0.2284
-0.2284
-0.2309
-0.2309
-0.2330
-0.2330
-0.2349
-0.2349
-0.2363
-0.2363
-0.2363
-0.2113
-0.2113
-0.2081
-0.2081
-0.2058
-0.2058
-0.2040
-0.2024
-0.2024
-0.2012
-0.1999
-0.1999
-0.1989
-0.1989
-0.1980
-0.1971
-0.1964
-0.1957
-0.1949
-0.1943
-0.1936
-0.1936
-0.1931
-0.1923
-0.1916
-0.1916
-0.1912
-0.1906
-0.1906
-0.1900
-0.1900
-0.1894
-0.1887
-0.1883
-0.1876
-0.1876
-0.1870
-0.1864
-0.1861

-0.1861
-0.1853
-0.1848
-0.1842
-0.1837
-0.1832
-0.1832
-0.1826
-0.1820
-0.1815
-0.1809
-0.1809
-0.1805
-0.1805
-0.1798
-0.1798
-0.1792
-0.1786
-0.1781
-0.1775
-0.1769
-0.1763
-0.1758
-0.1758
-0.1752
-0.1752
-0.1746
-0.1746
-0.1740
-0.1735
-0.1728
-0.1723
-0.1716
-0.1716
-0.1709
-0.1709
-0.1703
-0.1696
-0.1696
-0.1691
-0.1683
-0.1683
-0.1677
-0.1677
-0.1670
-0.1663
-0.1657
-0.1649
-0.1649
-0.1643
-0.1643
-0.1635
-0.1635
-0.1626
-0.1626
-0.1619
-0.1610
-0.1601
-0.1601
-0.1594
-0.1585
-0.1575
-0.1565
-0.1555

-0.1555
-0.1546
-0.1546
-0.1535
-0.1524
-0.1512
-0.1501
-0.1501
-0.1488
-0.1488
-0.1476
-0.1476
-0.1462
-0.1462
-0.1448
-0.1433
-0.1417
-0.1401
-0.1385
-0.1385
-0.1368
-0.1368
-0.1350
-0.1350
-0.1332
-0.1332
-0.1311
-0.1288
-0.1267
-0.1242
-0.1242
-0.1218
-0.1218
-0.1190
-0.1163
-0.1132
-0.1101
-0.1068
-0.1068
-0.1032
-0.0994
-0.0954
-0.0910
-0.0910
-0.0865
-0.0816
-0.0816
-0.0762
-0.0708
-0.0645
-0.0645
-0.0580
-0.0580
-0.0511
-0.0435
-0.0435
-0.0355
-0.0355
-0.0267
-0.0171
-0.0171
-0.0069
0.0004
0.0004

0.0004
0.0004
0.0003
0.0003
0.0003
0.0003
0.0003
0.0003
0.0003
0.0003
0.0003
0.0003
0.0002
0.0002
0.0003
0.0002
0.0002
0.0003
0.0003
0.0003
0.0002
0.0002
0.0003
0.0002
0.0002
0.0003
0.0003
0.0003
0.0001
0.0001
0.0004
0.0004
0.0003
0.0003
0.0002
0.0002
0.0003
0.0003
0.0003
0.0003
0.0003
0.0002
0.0002
0.0002
0.0003
0.0003
0.0002
0.0002
0.0002
0.0002
0.0002
0.0003
0.0003
0.0002
0.0004
0.0004
0.0003
0.0003
0.0002
0.0002

0.0002
0.0003
0.0003
0.0002
0.0002
0.0002
0.0002
0.0003
0.0003
0.0002
0.0003
0.0002
0.0002
0.0003
0.0003
0.0001
0.0002
0.0002
0.0003
0.0003
0.0001
0.0001
0.0001
0.0001
0.0002
0.0002
0.0001
0.0001
0.0001
0.0002
0.0004
0.0002
0.0002
0.0002
0.0003
0.0003
0.0002
0.0002
0.0002
0.0003
0.0003
0.0002
0.0001
0.0001
0.0001
0.0002
0.0002
0.0003
0.0003
0.0001
0.0001
0.0001
0.0002
0.0003
0.0003
0.0001
0.0001
0.0001
0.0001
0.0001
0.0002
0.0002
0.0002
0.0003

```
0.0001
0.0000
0.0000
0.0000
0.0000
0.0001
0.0001
0.0001
0.0002
0.0003
0.0003
0.0001
0.0001
0.0001
0.0001
0.0001
0.0002
0.0002
0.0001
0.0001
-0.0000
-0.0001
-0.0002
-0.0002
-0.0001
-0.0001
-0.0000
0.0000
0.0002
0.0002
0.0003
0.0001
0.0001
0.0001
0.0000
0.0000
0.0001
0.0001
0.0002
0.0002
0.0003
0.0001
0.0001
-1.5854
```

```
disp(sum_of_squared_errors)
```

```
36.5655
```

```
disp(root_mean_squared_error)
```

```
0.2400
```

```
disp(R2_value)
```

```
0.4199
```

Task 2: Plot the electrical terms that express how energy moves in a circuit

To visualize the battery's charging policy, write code to visualize the current, voltage, and power measurements for one charging cycle. Present the visualization as one figure with three subplots, one each for:

- Voltage vs. time
- Current vs. time
- Power vs. time

```
% Insert your code in the spaces below (or make use of helper functions and
% indicate where they can be found). Ensure your code is well-documented.

% STEP 1
% Create a figure handle and initialize the first subplot. Plot voltage vs.
% time.

%chosen cycle I want to focus on (has been the standard so far
cycle_number = 1;

%getting only the segmented data for the chosen cycle
cycle_data = segmentedRawDataTable(segmentedRawDataTable.Cycle_Index ==
cycle_number,:);

%resets time so the cycle data starts at the 0 second mark
cycle_data.DateTime = seconds(cycle_data.DateTime - cycle_data.DateTime(1));

%defining variables to make them easier to input
time = cycle_data.DateTime;
current = cycle_data.Current;
voltage = cycle_data.Voltage;

%creating figure
figure_handle = figure('Name', 'Battery Charging Profile', 'Position',
[200,100,800,900]);

%creating a 3 row by 1 column matrix for each of the graphs
% (Voltage v time, Current v time, and power v time,
% this one is first subplot indicated by the third

subplot(3, 1, 1)

%importing from the helper function(CycleModes) so we don't have to keep
%rewriting the same function over again
plotcycle_modes(time, voltage, cycle_data.CyclingModes)

%also plotting the original function for Voltage v time on the same graph
%(under it where it is black and dotted line) (idk if this is needed)
hold on
plot(time, voltage, 'k', 'LineStyle','--')
hold off
```

```

%titling the first subplot
title('Voltage vs. Time', 'FontSize', 16)
xlabel('Time (s)')
ylabel('Voltage (V)')

% STEP 2
% Use a second subplot to plot current vs. time.

%repeating the same steps as done in step 1
subplot(3,1,2)
plotcycle_modes(time, current, cycle_data.CyclingModes)

hold on
plot(time, current, 'k', 'LineStyle','--')
hold off

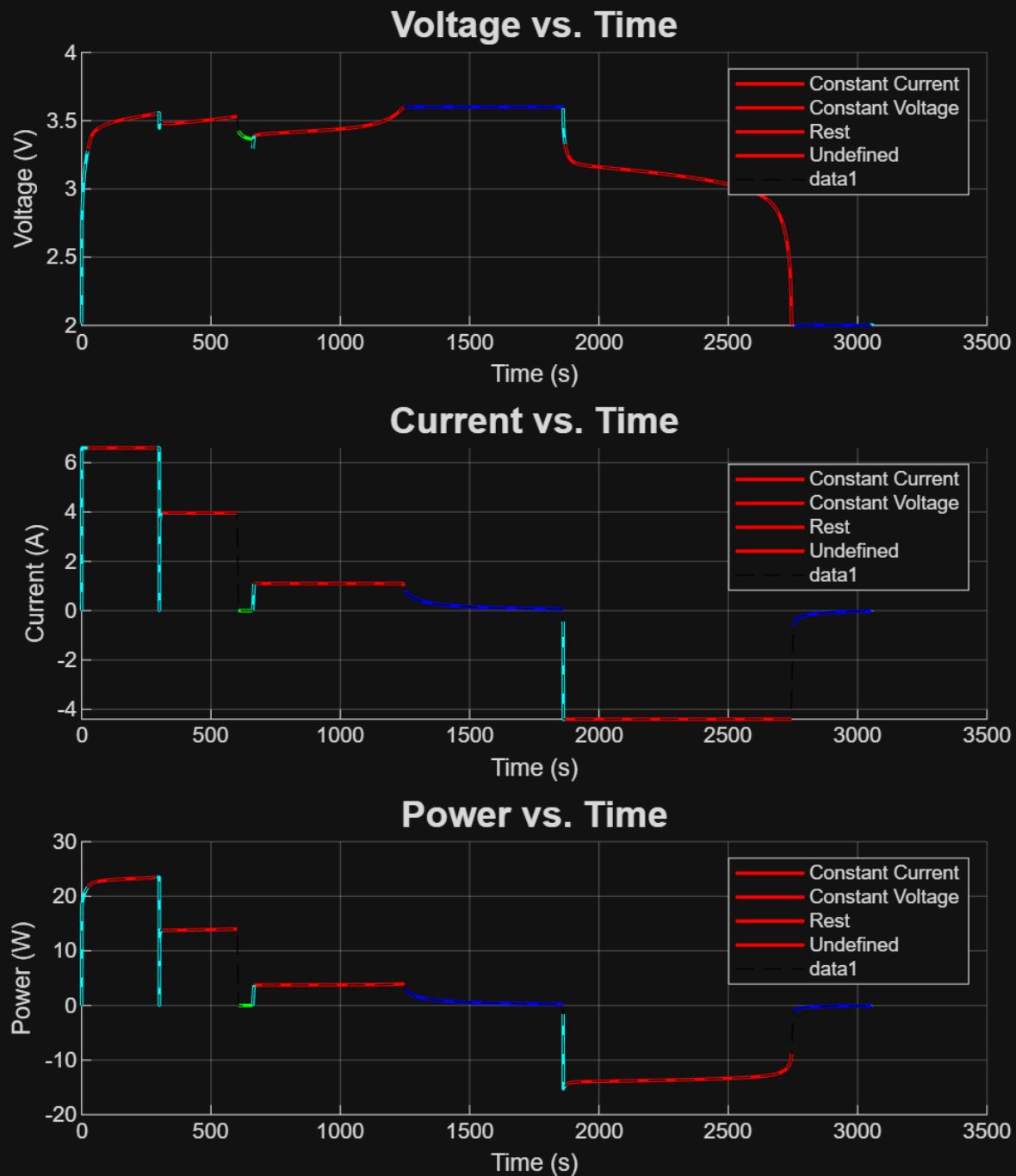
title('Current vs. Time', 'FontSize', 16)
xlabel('Time (s)')
ylabel('Current (A)')

%I DONT KNOW WHY EVERYTHING IN THE LEGEND SHOWS RED

% STEP 3
% Use a third subplot to plot power vs time.
% Hint:  $P = I * V$  where I is current in amps. You will first
% need to compute power before you can create this plot.

%calculating power using  $P=IV$  (use . since multiplying rows in matrix)
power = current .*voltage;

```



Task 3: Compute analytical results

To get started on using MATLAB for numerical integration and differentiation, start with this [doc page](#).

- Conduct a rate-of-change analysis by computing the derivative at key intervals in the voltage data
- Compute the time required to reach 80% and 100% charge

- Compute the total energy delivered to the battery by using integration to calculate the area under the power-time curve.
- Estimate the resistive energy loss
- Save the results to a summary table.

```
% Insert your code in the spaces below (or make use of helper functions and
% indicate where
% they can be found). Ensure your code is well-documented.

% STEP 1: Compute the rate of voltage change at different key time
% intervals. This helps you mathematically characterize the battery's
% charging profile and is more reliable than simply eyeballing the voltage
% vs. time plot.
% Optional: plot the points of notable transitions in charging dynamics in
% the voltage vs. time plot generated above
% Hint: explore using the gradient function

% STEP 2: Compute the time required to reach 80% charge and 100% charge

% STEP 3: Use integration to calculate the area under the power-time curve.
% This represents the total energy delivered to the battery during charging.
% Hint: explore using the trapz function

% STEP 4: Estimate the resistive energy loss.
% Hint: use Joule's heating formula ( $P = I^2 * R$ ) to estimate power loss in
% watts. The energy loss is given as  $E = P * t$ , where  $t$  is time in seconds
% and energy is in joules.

% STEP 5: Save results to a summary table. Display the table.
```

Interpretation of Results

Summarize your findings by interpreting their physical/engineering meaning (for example, why would engineers want to compute the analytical results in Task 3?):

What are some limitations of your work?

What are practical next steps?